CO4_Assessment 1- Code Challenge

Name:- Y.Thanushma

Reg. Number:-192425355

Course Code:-CSA0617 - Design and Algorithm Analaysis

Q1. Unique Paths with Obstacles (Dynamic Programming)

Problem Statement:

A grid contains obstacles that block movement, and a robot must move from the top-left corner to the bottom-right corner. The robot can move only right or down. The goal is to count the number of valid paths while avoiding obstacles. Use Dynamic Programming to solve the problem efficiently.

Python Program:

```
def unique_paths(grid):
    m = len(grid)
    n = len(grid[0])
    dp = [[0 for _ in range(n)] for _ in range(m)]
    if grid[0][0] == 1:
        return 0
    dp[0][0] = 1
    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                dp[i][j] = 0
            else:
                if i > 0:
                    dp[i][j] += dp[i-1][j]
                if j > 0:
                    dp[i][j] += dp[i][j-1]
    return dp[m-1][n-1]
```

```
grid = []
m = int(input("Enter rows (m): "))
n = int(input("Enter columns (n): "))
print("Enter the grid (0 = free, 1 = obstacle):")
for i in range(m):
    grid.append(list(map(int, input().split())))
print("Paths =", unique_paths(grid))
```

Sample Input

Enter rows (m): 3

Enter columns (n): 3

Enter the grid (0 = free, 1 = obstacle):

0 0 0

0 1 0

0 0 0

Sample Output

Paths = 2

Analysis

- Dynamic Programming stores the number of valid paths to each cell.
- An obstacle cell (1) cannot be visited, so its path count is set to 0.
- For a free cell (0), the number of paths is the sum of paths from the top and left cells.
- The obstacle in the center blocks paths passing through it.
- Time Complexity: O(m × n)
- Space Complexity: O(m × n)

Conclusion

Dynamic Programming efficiently calculates the number of valid paths by storing previously computed results and avoiding paths that pass through obstacles. For the given 3 × 3 grid, there are 2 valid paths from the start to the destination.

Execution:

```
File  Edit  Format  Run  Options  Window  Help
def unique_paths(grid):
    m = len(grid)
    n = len(grid[0])

    dp = [[0 for _ in range(n)] for _ in range(m)]

    if grid[0][0] == 1:
        return 0

    dp[0][0] = 1

    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                dp[i][j] = 0
            else:
                if i > 0:
                    dp[i][j] += dp[i-1][j]
                if j > 0:
                    dp[i][j] += dp[i][j-1]

    return dp[m-1][n-1]


grid = []

m = int(input("Enter rows (m): "))
n = int(input("Enter columns (n): "))

print("Enter the grid (0 = free, 1 = obstacle):")

for i in range(m):
    grid.append(list(map(int, input().split())))

print("Paths =", unique_paths(grid))
```

Output:

```
==========================================
Enter rows (m): 3
Enter columns (n): 3
Enter the grid (0 = free, 1 = obstacle):
0 0 0
0 1 0
0 0 0
Paths = 2
```

Q2. Minimum Falling Path Sum (Dynamic Programming)

Problem Statement

Given a square matrix, find the minimum sum of a falling path from the top row to the bottom row. At each step, the element can move directly below, diagonally left, or diagonally right. The objective is to determine the minimum possible sum using Dynamic Programming.

Python Program

```
def min_falling_path_sum(matrix):
    n = len(matrix)
    dp = [[0 for _ in range(n)] for _ in range(n)]
```

```
    for j in range(n):
        dp[0][j] = matrix[0][j]
    for i in range(1, n):
        for j in range(n):
            dp[i][j] = matrix[i][j] + dp[i-1][j]
            if j > 0:
                dp[i][j] = min(dp[i][j], matrix[i][j] + dp[i-1][j-1])
            if j < n-1:
                dp[i][j] = min(dp[i][j], matrix[i][j] + dp[i-1][j+1])
    return min(dp[n-1])
n = int(input("Enter matrix size (n): "))
matrix = []
print("Enter the matrix:")
for i in range(n):
    matrix.append(list(map(int, input().split())))
print("Min Sum =", min_falling_path_sum(matrix))
```

Sample Input

Enter matrix size (n): 3

Enter the matrix:

2 1 3

6 5 4

7 8 9

Sample Output

Min Sum = 13

Analysis

- Dynamic Programming stores the minimum falling path sum for each cell.
- For each cell, the minimum value is obtained from the top, top-left, or top-right cell.
- The current matrix value is added to the minimum of these possible previous paths.

- The minimum value in the last row gives the final answer.
- Time Complexity: O(n × n)
- Space Complexity: O(n × n)

Conclusion

Dynamic Programming efficiently finds the minimum falling path sum by storing the minimum cost of reaching each cell and avoiding repeated calculations. For the given matrix, the minimum falling path sum is 13.

Execution:

```
def min_falling_path_sum(matrix):
    n = len(matrix)
    dp = [[0 for _ in range(n)] for _ in range(n)]

    for j in range(n):
        dp[0][j] = matrix[0][j]

    for i in range(1, n):
        for j in range(n):
            dp[i][j] = matrix[i][j] + dp[i-1][j]

            if j > 0:
                dp[i][j] = min(dp[i][j], matrix[i][j] + dp[i-1][j-1])

            if j < n-1:
                dp[i][j] = min(dp[i][j], matrix[i][j] + dp[i-1][j+1])

    return min(dp[n-1])


n = int(input("Enter matrix size (n): "))
matrix = []

print("Enter the matrix:")

for i in range(n):
    matrix.append(list(map(int, input().split())))

print("Min Sum =", min_falling_path_sum(matrix))
```

Output:

```
================================
Enter matrix size (n): 3
Enter the matrix:
2 1 3
6 5 4
7 8 9
Min Sum = 13
>
```

Q3. Maximum Length Chain of Pairs (Dynamic Programming)

Problem Statement

Given a set of pairs of numbers, where each pair contains a start and end value, form the longest possible chain such that the end value of one pair is less than the start value of the next pair. Use Dynamic Programming to find the maximum chain length.

Python Program

```
def max_chain_length(pairs):
    n = len(pairs)
    pairs.sort(key=lambda x: x[0])
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if pairs[j][1] < pairs[i][0]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
n = int(input("Enter number of pairs (n): "))
pairs = []
print("Enter the pairs:")
for i in range(n):
    a, b = map(int, input().split())
    pairs.append((a, b))
print("Max Chain Length =", max_chain_length(pairs))
```

Sample Input

Enter number of pairs (n): 3

Enter the pairs:

5 24

15 25

27 40

Sample Output: Max Chain Length = 2

Analysis

- Dynamic Programming stores the maximum chain length ending at each pair.
- The pairs are first sorted according to their start value.
- A pair can be added to the chain if its end value is less than the start value of the current pair.
- dp[i] represents the maximum chain length ending with pair i.
- Time Complexity: $O(n^2)$
- Space Complexity: O(n)

Conclusion

Dynamic Programming efficiently finds the longest chain by comparing each pair with the previously considered pairs. For the given input, the maximum chain length is 2, such as (5,24) → (27,40).

Execution:

File Edit Format Run Options Window Help

```
def max_chain_length(pairs):
    n = len(pairs)

    pairs.sort(key=lambda x: x[0])

    dp = [1] * n

    for i in range(n):
        for j in range(i):
            if pairs[j][1] < pairs[i][0]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)


n = int(input("Enter number of pairs (n): "))
pairs = []

print("Enter the pairs:")

for i in range(n):
    a, b = map(int, input().split())
    pairs.append((a, b))

print("Max Chain Length =", max_chain_length(pairs))
```

Output:

```
================================
Enter number of pairs (n): 3
Enter the pairs:
5 24
15 25
27 40
Max Chain Length = 2
```